

Yep — that's the right stopping point. The variable-name theory is conclusively dead: EZ-BASIC recognizes variable names through 10 characters, so `EAT1`, `EAF1`, `EFY`, `EFY1`, etc. are distinct.

BASIC Studio Reference Manual -...

At this point I would **not invent another theory**. The current build is already instrumented, so the next session should start by reading the RUN diagnostics from build

`612837b007ec45db` and only follow the first concrete failure they expose.

Fresh-session handoff. Do not begin by changing code.

Current build:

`612837b007ec45db`

CURRENT SYMPTOM

=====

Enemy animation/facing is still wrong.

The user's painted FRONT frames exist, but the running enemy has been showing the wrong/static sprite behaviour.

A large number of downstream theories have already been tested.

Most importantly, the latest hypothesis is now DISPROVED:

EZ-BASIC variable names are significant through 10 characters.

Therefore:

`EAT1 != EAF1`

`EFY != EFY1`

`ESP1 != ESR1`

`EWK1 != EWH`

Do NOT rename these variables or convert them to arrays as a bug fix.

Arrays may still be useful later for code cleanup, but they are not the cause.

=====

WHAT IS ALREADY VERIFIED

=====

- allocator produces 115 cells
- atlas coordinates have been checked
- identifiers bind
- generated program parses cleanly
- TDZ/static JS checks are clean
- variable names are legal
- frame lookup assertions exist

- residual-label assertion exists
- leading-zero-expression assertion exists
- unknown-kind assertion exists
- ETR-survival assertion exists
- buildDoomBAS is already instrumented

The previous animation/layout work DID cause a separate real problem:

```
manifest/frame changes
  ->
atlas reflow
  ->
old positional painted pixels became orphaned
```

That architectural issue is being addressed separately with semantic/content-addressed sprite art.

Do not conflate that with the remaining runtime animation/facing bug unless the diagnostics specifically point there.

```
=====
FIRST ACTION: READ RUN OUTPUT
=====
```

Before editing anything:

1. Confirm the running app reports build:

```
612837b007ec45db
```

2. Press RUN.

3. Capture/report EVERY diagnostic emitted by buildDoomBAS.

Specifically look for:

```
[DOOM] ... SPRTEX line(s) still reference ETR
unsubstituted markers
unknown resource kinds
enemy front was never substituted
gun frame lookup failures
residual line labels
=0+ hardware syntax errors
undocumented function calls
any thrown exception from buildDoomBAS
```

Do not paraphrase them.

Return the exact console lines.

```
=====
WHY THIS IS THE PRIORITY
```

=====

The current generator has assertions specifically designed to separate:

- A. frame lookup failed
- B. template substitution failed
- C. final BASIC still contains obsolete coordinates/variables
- D. generator threw before completing
- E. generated BASIC is clean, meaning the remaining bug is runtime state/math

We should not create another theory until we know which category actually applies.

=====

IF ETR ASSERTION FIRES

=====

If any final SPRTEX still contains:

ETR

then inspect only those lines.

Report:

enemy id
facing/animation
frame name requested
findCell result
page/rect
_painted result
template regex match result

Do not modify EVA/facing maths.

=====

IF "ENEMY FRONT WAS NEVER SUBSTITUTED" FIRES

=====

Inspect enemy 1's actual resource id and resolve:

front1
front2
front3
front4

Report each:

```
cell
page
rect
spritePages buffer present yes/no
painted yes/no
```

Then report the resulting FRONT sequence.

```
=====
IF ALL BUILD ASSERTIONS ARE CLEAN
=====
```

Only then move to runtime diagnostics.

For enemy 1 display/log:

```
EVA
EAF1
EAT1
EHM
selected facing/frame
```

The current intended facing rule is:

```
EVA > 0.5      -> FRONT
EVA < -0.5     -> BACK
otherwise      -> SIDE
```

The current animation index should cycle:

```
EAF1 = 0,1,2,3
```

For three painted FRONT frames the generator should map:

```
0 -> front1
1 -> front2
2 -> front3
3 -> front1
```

If EAF1 changes but the image does not:
trace SPRTEX runtime writes.

If EAF1 remains 0:
inspect animation-counter execution.

If EVA never enters FRONT:
inspect facing-vector maths.

Do not infer which one before measuring.

```
=====
FINAL BASIC INSPECTION
```

=====

If build diagnostics are clean, dump the FINAL generated BASIC and extract all lines containing:

EVA
EAT1
EAF1
SPRTEX 1,

We need the final program, not the template.

Confirm that FRONT animation lines literally exist in the exported program.

=====

DO NOT CHANGE

=====

Do not touch:

- BSIM/BSPL writers
- DDA
- WXF/TXC
- wall projection
- atlas allocation
- z ordering
- variable naming
- animation architecture
- sprite dimensions

Do not introduce another speculative fix before reading the existing instrumentation.

=====

REPORT FIRST

=====

Return, in this order:

1. confirmed running build hash
2. exact RUN console diagnostics
3. whether any ETR survived
4. whether enemy-front substitution assertion fired
5. whether any marker/label/syntax assertion fired
6. whether buildDoomBAS threw
7. only if all clean: runtime EVA/EAF/EAT values
8. exact final SPRTEX 1 lines

Follow the first measured broken link.

Do not guess beyond it.

That's the cleanest reset: **one build, one RUN, read the instrumentation already there.**

The manual check was still worthwhile because it removes a very plausible hardware-only

mode without another PS2 test cycle.

Sources